

Setting Up Continuous Integration and Delivery (CI/CD) Service

SAP Continuous Integration and Delivery (CI/CD) is a service on SAP BTP that enables you to configure and run predefined continuous integration and delivery pipelines. It connects with your Git repository and builds, tests, and deploys your code changes.

SAP Continuous Integration and Delivery has, among others, a ready-to-use pipeline for SAP Fiori in the Cloud Foundry Environment. For further information, see [SAP Help Portal](#).

In this mission, you will use GitHub as a remote repository.

Step 1 - Check Entitlement and Instance of your CI/CD service

1. Go to your SAP BTP Enterprise or Trial Account and open the SAP BTP Cockpit.
2. Check your **Entitlements**. Choose **Entity Assignments** and check that you are entitled to use the BTP Service **Continuous Integration & Delivery (CI/CD)** in your subaccount.

It is a "shared unit", so no quota assignment for your subaccount is required.

3. Choose **Account Explorer** in the left-hand navigation pane and choose your **Subaccount** in which you want to run CI/CD.
4. In your Subaccount, choose **Instances and Subscriptions** and check if the Continuous Integration & Delivery service is already subscribed.

This is the case if you executed the booster in the previous section with the option to activate CI/CD.

Step 2 - Activate CI/CD service

1. If you did not already subscribe to the CI/CD service, choose **Service Marketplace** in the left-hand pane.
2. Type **Continuous Integration & Delivery** in the search box and choose the service tile.

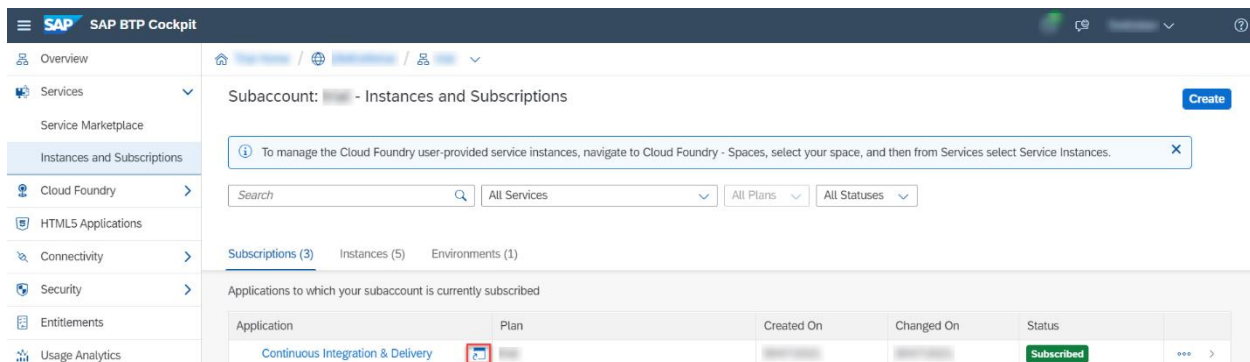
For **Service Plan** choose for an enterprise account **default - Subscription (enterprise)** or for a trial account choose **Subscription (trial)**.

3. Choose Create.
4. Choose Create in the pop-up without changing any values. Wait until the status changes to **Subscribed**.

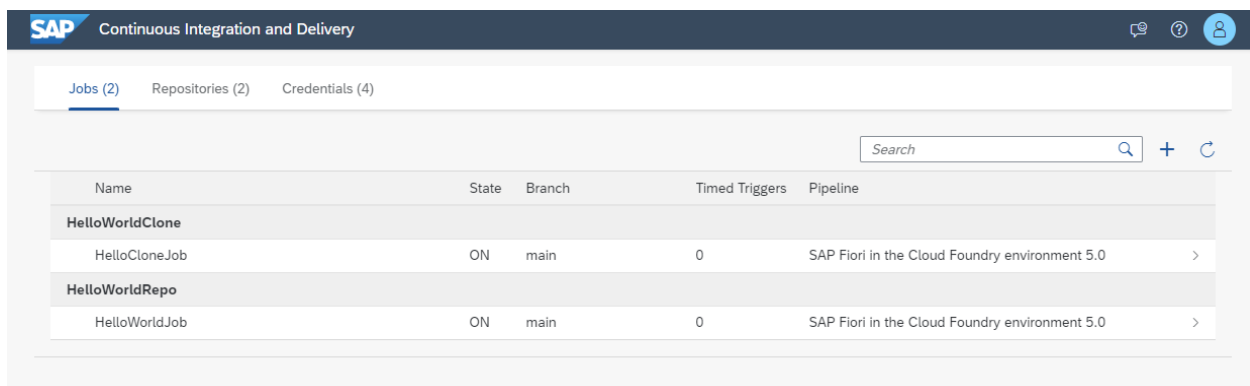
5. From the left-handed cockpit navigation pane, choose **Security** and then **Security → Users**.
6. Select your user. A detail screen for the user will open on the right-hand side.
7. In the detail screen, scroll to "**Role Collections**" and click **Assign Role Collection**.
8. From the dropdown list, select **CICD Service Administrator** and **CICD Service Developer** and confirm your choice with Assign Role Collection.

Step 3 - Access Continuous Integration and Delivery Service

1. In your SAP BTP subaccount, navigate to Services → Instances and Subscriptions in the left-hand pane.
2. Choose the Go to Application icon located next to the Continuous Integration & Delivery subscription.



3. Use your credentials to log in to the application if required.
4. You will see an empty Continuous Integration and Delivery service. You will create two jobs in this tutorial for two remote repositories.



Step 4 - Create Credentials for Accessing GitHub and BTP

1. Choose **Credentials** on the top-level navigation.
2. Click in the "+" icon, **Create Credential**.
3. Create Credentials for accessing the GitHub repo.
 - Credentials Name: any
 - Username: your GitHub user name
 - Password: your GitHub password. You may have to use a Personal Access Token.



The screenshot shows a 'Create Credentials' form with the following fields and values:

- *Credentials Name:** githubhelloworld
- Description:** (empty)
- Type:** Basic Authentication (selected from a dropdown menu)
- *Username:** myemail@company.com
- Password:** (masked with dots, with a toggle icon to show/hide)

At the bottom right, there are two buttons: **Create** and **Discard**.

4. Create Credentials for accessing the BTP platform, which will be needed to deploy the build archive.
 - Provide a name for the entry, e.g., "deployonbtp."
 - Choose Authentication Type, e.g., Basic Authentication
 - Provide the user and password for a user who has the assigned role collection for CI/CD usage.
 - For productive usage, consider creating and using a technical user.

Step 5 - Create a CI/CD Entry for your GitHub Repository

1. Click on "Repositories" in the top-level navigation.
2. Click in the "+" icon, **Add repository**.
3. Provide the required information, e.g.:
 - Name: e.g. HelloWorldRepo

- Clone URL: your GitHub Repo URL, which contains the **helloworldui5** project from the previous section.
- Credentials: the one we just defined

Generate a **WebHook Event Receiver**: Choose "Generate"

The screenshot shows the 'Add Repository' configuration page. On the left is a sidebar with a search bar and a list of items. The main area has a title 'Add Repository' with a GitHub icon. Below the title is a tab 'General Information'. The form fields are as follows:

Field	Value
*Name:	HelloWorldRepo
*Clone URL:	https://github.com/org/btp-helloworld-app.git
Credentials:	githubtoolshelloworld
Cloud Connector:	

Below the 'General Information' section is the 'WEBHOOK EVENT RECEIVER' section. It contains the following fields:

Field	Value
Type:	GitHub
*Webhook Credential:	<GENERATE>
State:	ON

A 'Remove' button is located in the top right corner of the 'WEBHOOK EVENT RECEIVER' section.

4. Check the generated Webhook Data. Click on Webhook Data.

You will need this Data for your GitHub Repository and the CI/CD pipeline.

Webhook Creation

Navigate to the GitHub repository [HelloWorldRepo](#) and add webhook manually using the data below if not yet done. A webhook enables the repository to start builds.

Webhook Data

Payload URL: `https://cicd-service.cfapps. .hana.ondemand.com/v2/github_events/tenant/b9c7024ea-451c ed5401/webhook/9940929a-4-b2e6-2d3fecf` 

Content type: `application/json`

Secret: `2ccf0d1 2332e688960 /c732ce898ac56126038f` 

[Close](#)

Step 6 - Create the Webhook Credentials for the CI/CD pipeline

1. Click on Credentials again, choose "+" Add Credentials.
2. Provide the data, using the secret from your repository definition.

Edit Credentials

*Credentials Name:

Description:

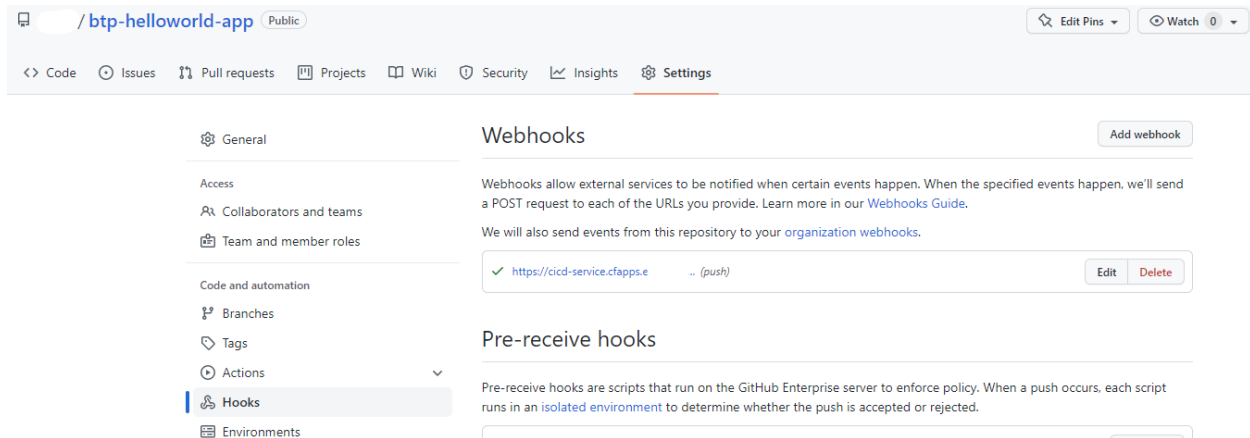
Type: 

*Secret: 

[Save](#) [Discard](#)

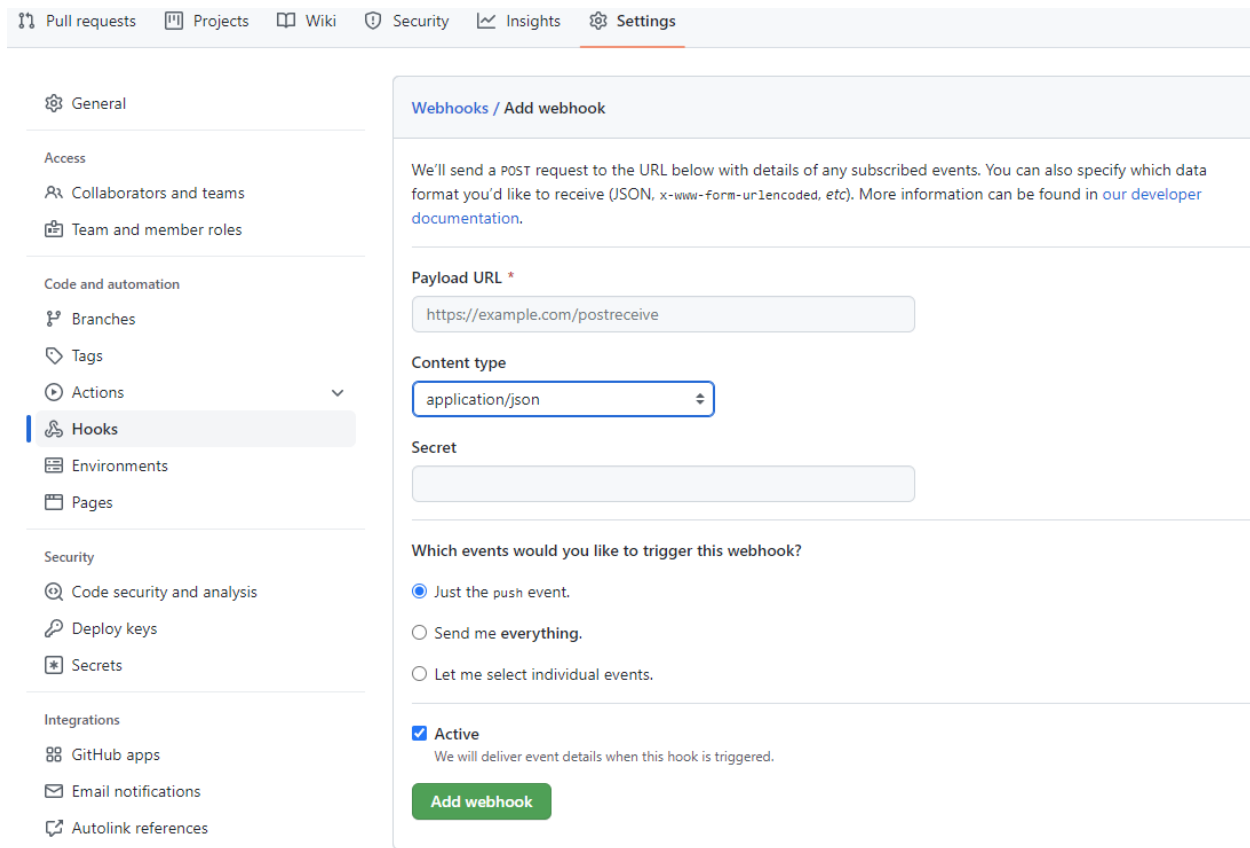
Step 7 - Create the Webhook in your GitHub Repository

1. Open your GitHub Repository, which contains your **helloworldui5 project** from the previous section.
2. Click on **Settings** and then on **Hooks**.



3. Click on **Add Webhook**.

Copy Payload URL and Secret from your generated Webhook Data from your CI/CD repository definition. SSL verification will be auto-enabled.



4. Click on **Add Webhook** and check the result.

Step 8 - Create a Job with your CI/CD Service Job Editor

1. Click on **Jobs** in the CI/CD service. Choose "+" **Create Job**.

2. Provide the **General Information**:

- Job Name: Any
- Repository: The one you just defined.
- Branch: **main** (typically main, if you did not change it)

- Pipeline: **SAP Fiori in the Cloud (this is a predefined pipeline)**

3. **STAGES**

- Configuration Mode: **Job Editor**
- Build Tool Version: Java 11 Node 16

Choosing Job Editor means you will use a form to provide the necessary pipeline data, not a .pipeline/config.yml file.

Create Job

Run ?

General Information Build Retention Stages Build Notification

*Job Name: HelloWorldJob

Description: describe it

*Repository: HelloWorldRepo

*Branch: main

Pipeline: SAP Fiori in the Cloud Foundry enviro... ?

Version: 5.0

State: ON

BUILD RETENTION

*Keep logs for: 7 days

*Keep maximum: 50 build items

STAGES

Configuration Mode: Job Editor

4. Skip some Stages

Switch OFF the following stages, because it is not part of a beginner tutorial.

- Addition Unit Tests
- Malware Scan
- Compliance

5. Switch **ON** your **Release** Stage. Provide the deployment information to your Cloud Foundry Space.

- **API Endpoint:** copy from your Subaccount "Overview" page

- **Org Name:** copy from your Subaccount "Overview" page
- **Space:** copy from your Subaccount "Overview" page
- **Deploy Type:** standard
- **Credentials:** deployonbtp or how you named it.

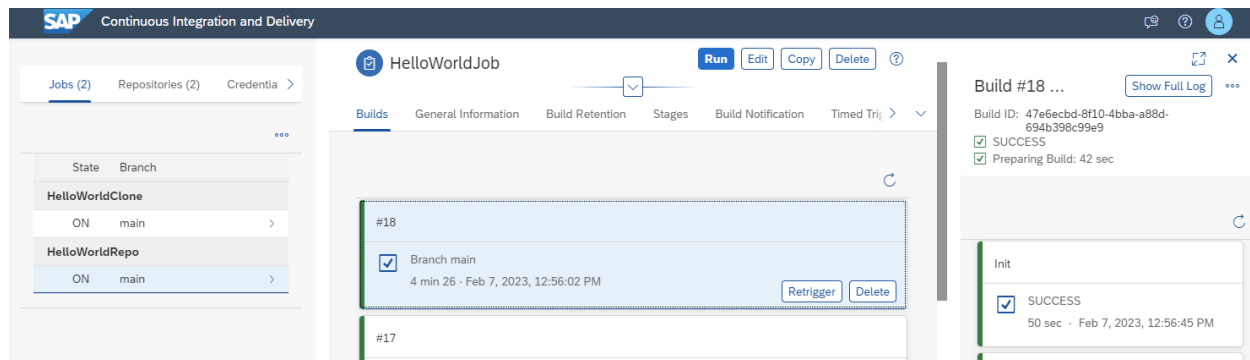
The screenshot shows the 'Create Job' configuration interface. At the top, there's a 'Create Job' header with a 'Run' button and a help icon. Below the header are four tabs: 'General Information', 'Build Retention', 'Stages' (which is selected), and 'Build Notification'. The 'Stages' tab contains two main sections: 'Release' and 'Deploy to Cloud Foundry Space'. The 'Release' section has fields for 'Token' and 'Credentials'. The 'Deploy to Cloud Foundry Space' section has a toggle switch set to 'ON' and several input fields: '*API Endpoint' (https://api.cf.eu.hana.ondemand.com), '*Org Name' (my_Cloud_Foundry_org_name), '*Space' (my_devspace_dev), '*Deploy Type' (standard), and '*Credentials' (deployonbtp). Below this is the 'Upload to Cloud Transport Management' section with a toggle switch set to 'OFF' and fields for 'Node Name' and 'Service Key'. At the bottom of each section are links for 'Additional Credentials' and 'Additional Variables'.

6. Click **Create**.

7. Click **Run**.

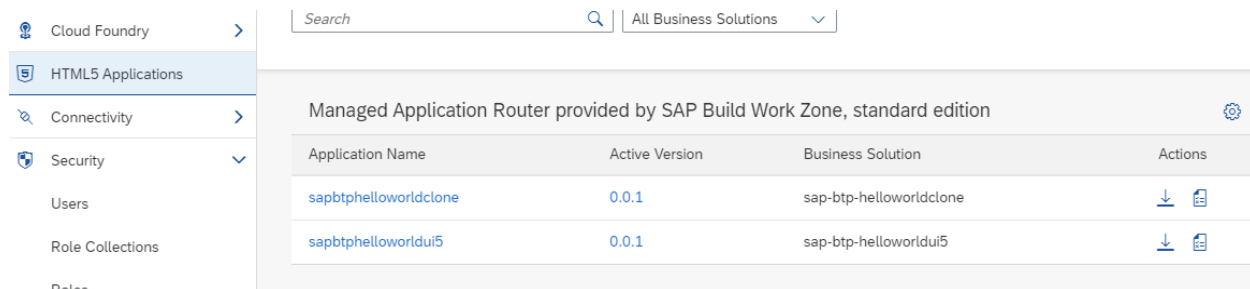
Build and deployment will take some minutes. You can see the log of the build process during execution of the build pipeline.

If everything is ok, you will get a green result log



8. Congratulations! If the Release Stage is also green, your Job has been deployed.

Check the result in your Subaccount **HTML5 Applications** : (Note: You can create the "helloworldclone" app in a different tutorial)



Step 9 - Use a config file instead of Job Editor

If you use subfolders in your Fiori project, you have to use **.pipeline/config.yml** pipeline definition in your Git repository.

You can create one from your Job Editor definitions.

Click on the small **YML icon** in your Job definition and an editor will open, containing your job definition as a YAML file.

If your subfolder is, for example, **helloworldclone**, you have to add in your "Build Stage" **source: helloworldclone**. E.g.:

stages:

Build:

source: helloworldclone

npmExecuteLint: false

The whole config.yml could look like this:

general:

buildTool: "mta"

service:

buildToolVersion: "MBTJ11N16"

stages:

Build:

source: helloworldclone

npmExecuteLint: false

Additional Unit Tests:

npmExecuteScripts: false

Malware Scan:

malwareExecuteScan: true

Acceptance:

cloudFoundryDeploy: false

npmExecuteEndToEndTests: false

Compliance:

sonarExecuteScan: false

Release:

cloudFoundryDeploy: true

cfApiEndpoint: "https://api.cf.eu10.hana.ondemand.com"

cfOrg: "mysubdomain"

cfSpace: "dev"

cfCredentialsId: "deployonbtppcredentials"

deployType: "standard"

tmsUpload: false

steps:

artifactPrepareVersion:

versioningType: "cloud_noTag"

cloudFoundryDeploy:

mtaDeployParameters: "-f --version-rule ALL"

You need to create a folder and file named **.pipeline/config.yml** in the root of your GitHub repository and provide the necessary configurations. Switch the **Configuration Mode** of your Job to **Source Repository** and **Run**.